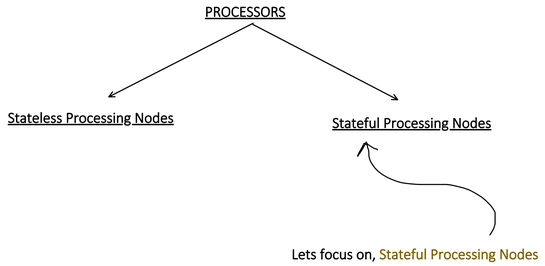
Kafka Stream: Stateful Processing Nodes

Wednesday, 22 July 2026 1:23 PM



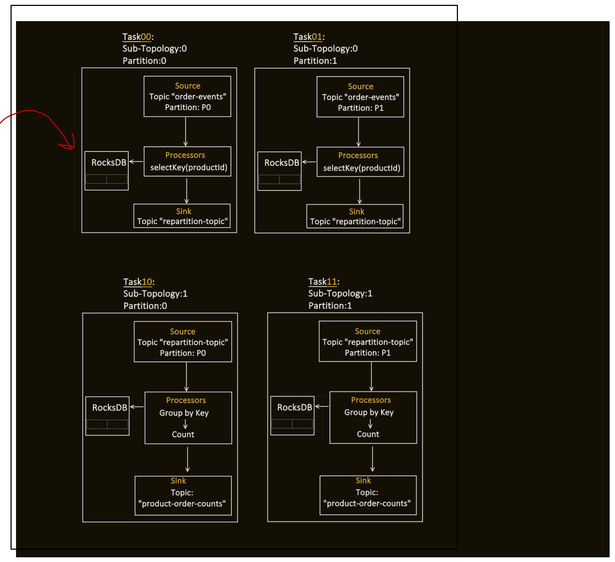
A stateful processing node is a node that must remember information from previous records to produce correct output.

Stateful operations use state stores (RocksDB) to store intermediate state.
Common examples:

- Count records per key
- Sum amounts per user

Any doubt with RocksDB, where does it fit, kindly check out Kafka Stream architecture topic





Important Stateful Operations:

We will discuss these:

1. ~~groupByKey~~
2. groupBy
3. count
4. aggregate
5. KStream-KTable join

Etc.

These 2 are not Stateful operations (as they do not store any intermediate state themselves) but provide entry point to stateful processing.

2 major responsibility of ***GroupbyKey*** and ***GroupBy***:

1. Convert KStream -----> KGroupedStream
This tells downstream stateful operations which key to use for aggregation (either existing key or some new key)

2. Repartition if required.
When key is changed, then it initiate the reparation.

1. groupByKey

- ***groupByKey*** tells downstream stateful operations to use existing key for aggregation.
- Use it when the current key is already the key you want for aggregation.

```
//optional, say in case you want to change the key, else if not don't use selectKey
KStream<String, Order> ordersByUser = orders.selectKey((orderId, order) -> order.getUserId());

KGroupedStream<String, Order> groupedOrders = ordersByUser.groupByKey();
```

This is not a normal Stream, it has method for aggregation like count, aggregate etc.

2. groupBy

groupBy says, I don't want to use the current key, I will tell you a new key (which could be of different type too)

```
KGroupedStream<String, Order> groupedOrders = orders.groupBy((orderId, order) -> order.getUserId());
```

For both ***groupByKey*** and ***groupBy***, if there is change in Key, Kafka Streams needs to redistribute records (re-partition) according to the new key.

3. count

- ***count*** counts the number of records for each key.

```
KTable<K, Long> count();
```

or

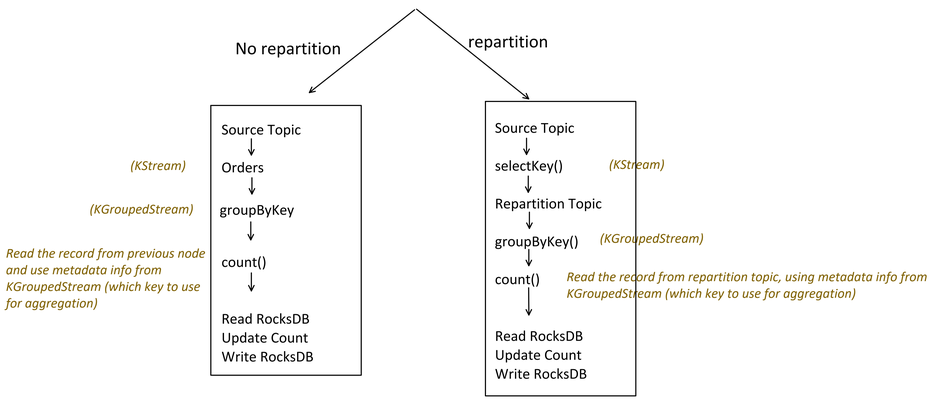
```
KTable<K, Long> count(Materialized<K, Long, KeyValueStore<Bytes, byte[]>> materialized);
```

```
KTable<String, Long> orderCount = orders
                                    .groupByKey()
                                    .count(
                                        Materialized.<String, Long, KeyValueStore<Bytes, byte[]>>as("order-count-store")
```

```
        .withKeySerde(Serdes.String())
        .withValueSerde(Serdes.Long())
);
```

We have already covered the SerDes (Serializer and Deserializer).






```
KTable<String, Long> orderCountByUser = orders
        .groupBy((orderId, order) -> order.getUserId())
        .count();
```

Input records:

Event	Order ID	User ID
1	o1	u1
2	o2	u1
3	o3	u2
4	o4	u1

State store after processing:

User ID	Count
u1	3
u2	1

4. aggregate

aggregate() → You decide how the aggregation should work.

```
<VR> KTable<K, VR> aggregate(Initializer<VR> initializer,
                             Aggregator<? super K, ? super V, VR> aggregator);
```

```
public interface Initializer<VAgg> {
    VAgg apply();
}
```

When we see the key for the first time, how to initialize it

```
public interface Aggregator<K, V, VAgg> {
    VAgg apply(K var1, V var2, VAgg var3);
}
```

Key	Value
Alice	Order(100)
Bob	Order(200)
Alice	Order(150)
Alice	Order(50)
Bob	Order(300)

Output we wanted is:

Alice - 300
Bob - 500

```
KGroupedStream<String, Order> groupedOrders = orders.groupByKey();

KTable<String, Double> totalAmount = groupedOrders.aggregate( () -> 0.0, (key, order, totalFromRocksDB) ->
                                                                totalFromRocksDB+ order.getAmount() );
```

When Alice comes, currently RocksDB has no key 'Alice'
So it will store:

Alice -> 0

Value present in RocksDB

Example:

Record: Alice -> 100

RocksDB lookup
↓
Not Found
↓
initializer() → 0
↓
Aggregator(Alice, Order(100), 0) → 100
↓
Store 100

Record: Alice -> 150

RocksDB lookup
↓
Found 100
↓
Aggregator(Alice, Order(150), 100) → 100+150
↓
Store 250

Record: Alice -> 50

RocksDB lookup
↓
Found 250
↓
Aggregator(Alice, Order(50), 250) → 250+50
↓
Store 300

5. KStream - KTable Join

A KStream-KTable join enriches each stream event with the latest value from a table.

```
KStream<K, VR> join(KTable<K, VT> table, ValueJoiner<? super V, ? super VT, ? extends VR> joiner);
```

```
public interface ValueJoiner<V1, V2, VR> {
    VR apply(V1 var1, V2 var2);
}
```

Enriched Output

KStream value

Latest value for the key from KTable

```
KStream<String, Order> ordersByUser = orders.selectKey((orderId, order) -> order.getUserId());

KTable<String, UserProfile> users = builder.table( "users",
                                                   Consumed.with(Serdes.String(), userProfileSerde),
                                                   Materialized .<String, UserProfile, KeyValueStore<Bytes, byte[]>>as("users-store")
                                                           .withKeySerde(Serdes.String())
                                                           .withValueSerde(userProfileSerde)
                                                 );

KStream<String, EnrichedOrder> enrichedOrders = ordersByUser.join(
                                                            users,
                                                            (order, user) -> new EnrichedOrder(order, user)
                                                          );
```

Note:

Key for both KStream and KTable should be same.

Example:

User table:

Key	Value
u1	{ name: "Asha", city: "Delhi" }
u2	{ name: "Ravi", city: "Pune" }

Order stream:

Key	Value
u1	{ orderId: "o1", userId: "u1", amount: 500 }

Output:

Key	Value
u1	{ orderId: "o1", userId: "u1", amount: 500, name: "Asha", city: "Delhi" }

Similarly we can do KTable - KTable Join

```
KTable<String, UserProfile> profiles = builder.table("user-profiles");
KTable<String, UserPreference> preferences = builder.table("user-preferences");

KTable<String, CompleteUser> completeUsers = profiles.join(
                                                preferences,
                                                (profile, preference) -> new CompleteUser(profile,
                                            preference)
                                        );
```

KTable - KStream : NOT POSSIBLE

A **KStream** is an unbounded sequence of events and cannot be randomly looked up.